

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ИНДИВИДУАЛЬНОЕ ДОМАШНЕЕ ЗАДАНИЕ
по дисциплине «Введение в нереляционные базы данных»
Тема: База данных актёров

Студенты гр. 3342

Преподаватель

Пушко К.Д.
Лапшов К.Н.
Антипина В.А.

Заславский М.М.

Санкт-Петербург

2026

ЗАДАНИЕ

Студенты

Лапшов К.Н.

Пушко К.Д.

Антипина В.А.

Группа 3342

Тема проекта: Разработка приложения для работы с базой данных актёров.

Исходные данные:

Необходимо разработать приложение для поиска, фильтрации и просмотра карточек актёров, используя базу данных MongoDB.

Содержание пояснительной записки:

«Введение»

«Сценарии использования»

«Модель данных»

«Разработанное приложение»

«Выводы»

«Приложения»

«Литература»

Предполагаемый объём пояснительной записки:

Не менее 10 страниц

Дата выдачи задания: 10.02.26

Дата сдачи реферата: 30.04.26

Дата защиты реферата: 30.04.26

	_____	Пушко К.Д.
	_____	Лапшов К.Н.
Студенты гр. 3342	_____	Антипина В.А.
Преподаватель	_____	Заславский М.М.

АННОТАЦИЯ

В рамках данного курса предполагалось разработать приложение на одну из заданных тем, использующее нереляционную базу данных, в команде из трёх человек. Была выбрана тема создания приложения для взаимодействия с базой данных актёров. Для хранения карточек артистов использовалась MongoDB. Исходный код хранится в репозитории проекта: <https://github.com/moevm/nsql1h26-actor>.

ABSTRACT

This course involved developing an application on one of the given topics using a non-relational database in a team of three. The chosen topic was to create an application for interacting with a database of actors. MongoDB was used to store the actor profiles. The source code is stored in the project repository: <https://github.com/moevm/nsql1h26-actor>.

Оглавление

1. Введение.....	7
2. Сценарии использования.....	8
3. Модель данных.....	15
4. Разработанное приложение.....	29
5. Выводы.....	32
6. Приложения.....	33
7. Литература.....	34

1. Введение

Актуальность решаемой проблемы.

Слова, сказанные В.И.Лениным в 1922 году о кинематографе не теряют актуальности и по сей день. «Из всех искусств для нас важнейшим является кино» - утверждал деятель, подчеркивая доступность этого жанра для простых людей и влияние, которое он оказывает на сознание массового зрителя. Исследование агентства стратегических инициатив (АСИ), проведённое в 2025 году, показало положительную динамику в отрасли: увеличилось число снятых фильмов, количество подписчиков онлайн-кинотеатров и зрителей отечественных картин. Это, безусловно, демонстрирует высокий спрос на продукцию индустрии. В производстве кино важную роль играет подбор актёров. Это порождает потребность в удобной платформе для поиска артистов, удовлетворяющих определённым критериям, необходимым для конкретного фильма. Разработанное в рамках этого курса приложение отвечает этим требованиям.

Постановка задачи.

Необходимо реализовать приложение для работы с базой данных актёров.

Предлагаемое решение.

Нужно реализовать веб-приложение, использующее MongoDB. Приложение позволит хранить карточки актёров и удобно с ними взаимодействовать.

Качественные требования к решению.

Должен поддерживаться массовый импорт/экспорт. Требуется функция фильтрации данных, поиска актёров, добавления новых сущностей.

2. Сценарии использования

Макет UI.

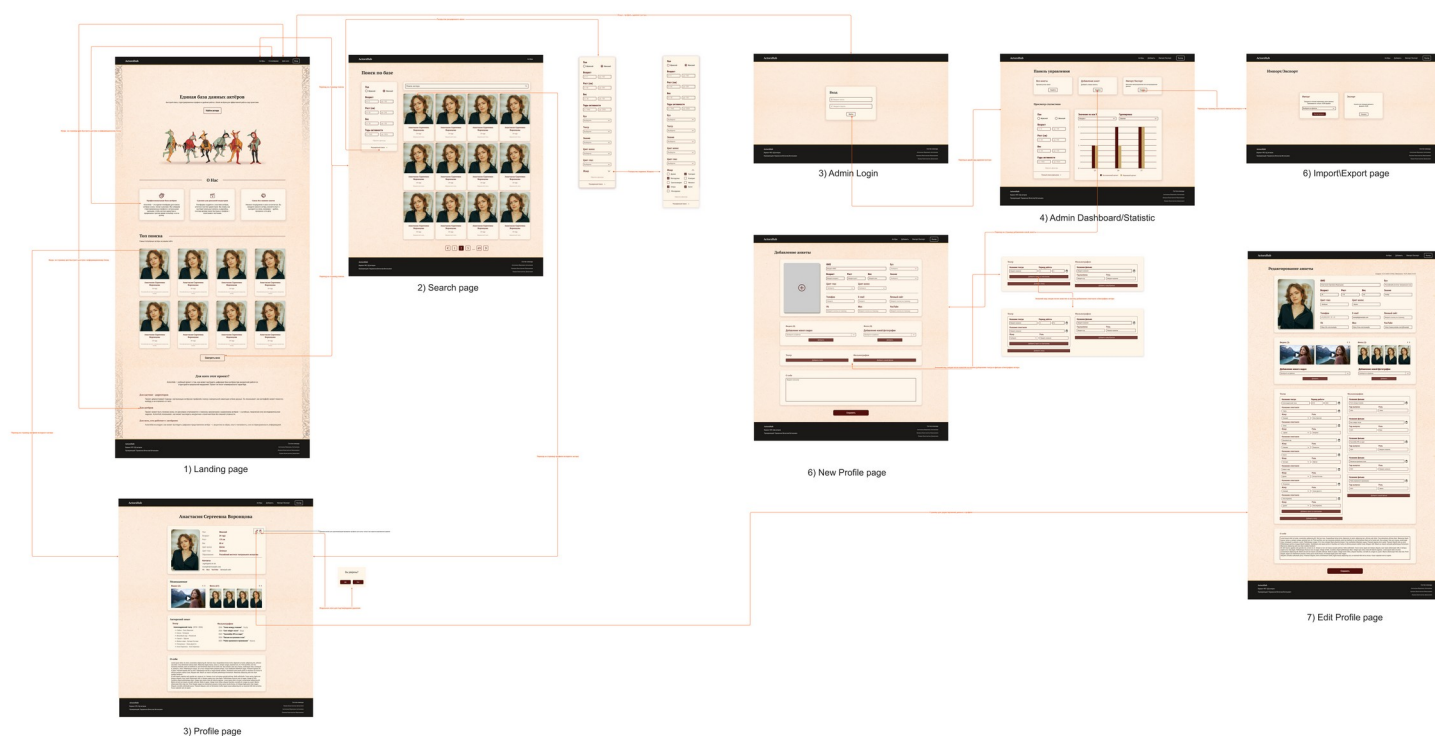


Рисунок 1 — все экраны приложения

Макет в Figma:

<https://www.figma.com/design/v0QiAXgQBAGjkuNp2ls4tA/Untitled?node-id=0-1&t=QR9byvO2Bb22TVyM-1>

Макет в репозитории:

<https://github.com/moevm/nsq11h26-actor/blob/123eab2fa3e09a894b3f31dd40a11a50a9132efc/docs/mockup.png>

Сценарии использования для задачи.

1. Импорта данных

Для массового импорта данных:

Предусловия: администратор авторизован

Основной сценарий:

Таблица 1 — основной сценарий импорта актёров

Шаг	Пользователь	Система
1	Переходит на страницу для	Система отображает

	Импорта/Экспорта (Import/Export page)	страницу
2	Нажимает «Импорт»	Система открывает окно выбора файла
3	Выбирает файл	Система загружает файл
4	Подтверждает импорт	Система создаёт профили
5	Завершение операции	Система отображает уведомление

Альтернативные сценарии:

Таблица 2 — альтернативный сценарий импорта актёров

Шаг	Пользователь	Система
1	Переходит на страницу для Импорта/Экспорта (Import/Export page)	Система отображает страницу
2	Нажимает «Импорт»	Система открывает окно выбора файла
3	Загружает файл с ошибками	Система выдаёт ошибку о невозможности загрузки

Для ручного задания данных:

Предусловия: администратор авторизован

Основной сценарий:

Таблица 3 — основной сценарий ручного создания карточки актёра

Шаг	Пользователь	Система
1	Открывает форму создания профиля	Система отображает форму
2	Загружает изображение	Система сохраняет изображение
3	Заполняет обязательные поля	Система валидирует данные
4	Заполняет дополнительные поля	-

5	Нажимает «Сохранить»	Система создаёт профиль
6	Успешное завершение	Система отображает новый профиль

Альтернативные сценарии:

Таблица 4 — альтернативный сценарий ручного создания карточки актёра

Шаг	Пользователь	Система
1	Открывает форму создания профиля	Система отображает форму
2	Загружает изображение	Система сохраняет изображение
3	Не заполняет обязательные поля	Система отображает ошибки

2. Представления данных

Просмотр лендинга и переход к поиску:

Предусловия:

- Пользователь не авторизован
- Открыта главная страница сервиса

Основной сценарий:

Таблица 5 — основной сценарий просмотра лендинга

Шаг	Пользователь	Система
1	Открывает сайт	Система отображает Landing page
2	Просматривает описание сервиса и карточки персон	-
3	Нажимает кнопку «Найти актёра»	Система переходит на страницу поиска

Альтернативные сценарии:

Таблица 6 — альтернативный сценарий №1

Шаг	Пользователь	Система
3	Нажимает кнопку «Смотреть всех» в блоке "Топ поиска"	Система переходит на страницу поиска

Таблица 7 — альтернативный сценарий №2

Шаг	Пользователь	Система
3	Нажимает кнопку «Актёры» в навигационной панели	Система переходит на страницу поиска

Поиск профилей:

Предусловия: открыта страница поиска

Основной сценарий:

Таблица 8 — основной сценарий поиска актёров

Шаг	Пользователь	Система
1	Выбирает все необходимые фильтры в блоке с фильтрами поиска	Система принимает параметры
2	Вводит ФИО желаемого актёра в поле поиска	Система выполняет поиск
3	Просматривает результаты	Система отображает список персон
4	Нажимает на карточку профиля	Система открывает страницу профиля

Альтернативные сценарии:

Таблица 9 — альтернативный сценарий №1

Шаг	Пользователь	Система
2	Запускает поиск без параметров	Система отображает все записи

Таблица 10 — альтернативный сценарий №2

Шаг	Пользователь	Система
3	Результаты не найдены	Система отображает сообщение «Ничего не найдено»

Просмотр профиля:

Предусловия: профиль существует.

Основной сценарий:

Таблица 11 — основной сценарий просмотра профиля

Шаг	Пользователь	Система
1	Просматривает основную информацию	-
2	Просматривает галерею	Система отображает изображения или видео
3	Читает биографию	-

3. Анализа данных

Просмотр статистики в панели администратора:

Предусловия:

- Администратор авторизован
- Открыта панель администратора (Admin Dashboard)

Основной сценарий:

Таблица 12 — основной сценарий просмотра статистики

Шаг	Пользователь	Система
1	Открывает Admin Dashboard	Система отображает панель управления
2	Просматривает блок «Просмотр статистики»	Система отображает фильтры и дефолтный график
3	Выбирает фильтры (пол, возраст, рост, вес, годы активности)	Система сохраняет выбранные значения и перестраивает график
4	Выбирает значение по оси X	Система принимает параметр оси X и перестраивает график

5	Выбирает параметр группировки	Система принимает параметр группировки и перестраивает график
6	Завершает выбор параметров	Система строит и отображает график статистики
7	Просматривает визуализированные данные	-

Альтернативные сценарии:

Таблица 13 — частичный выбор параметров

Шаг	Пользователь	Система
3	Выбирает только фильтры	Система строит график с учётом выбранных фильтров и установленными по умолчанию параметрами оси абсцисс и группировкой
4	Выбирает только ось X	Система строит график с выбранным параметром без фильтров и с установленной по умолчанию группировкой
5	Выбирает только группировку	Система строит график с выбранным параметром без фильтров и с установленной по умолчанию осью абсцисс

Таблица 14 — изменение параметров статистики

Шаг	Пользователь	Система
6	Изменяет любой фильтр	Система пересчитывает и обновляет график
7	Изменяет ось X или группировку	Система перестраивает график

Таблица 15 — сброс фильтров

Шаг	Пользователь	Система
1	Нажимает «Сбросить фильтры»	Система очищает выбранные параметры
2	Возврат к состоянию по умолчанию	Система перестраивает график на дефолтный

4. Экспорта данных

Предусловия: администратор авторизован

Основной сценарий:

Таблица 16 — основной сценарий экспорта данных

Шаг	Пользователь	Система
1	Выбирает профили	Система отмечает выбранные записи
2	Нажимает «Экспорт»	Система предлагает формат
3	Скачивает файл	Система завершает процесс

Вывод:

Для реализованного решения будут преобладать операции чтения, поскольку основное назначение приложение — просмотр профилей актёров. Правами добавления и редактирования карточек наделён только администратор и не подразумевается, что эти операции будут выполняться им часто.

3. Модель данных

Нереляционная модель данных.

Графическое представление

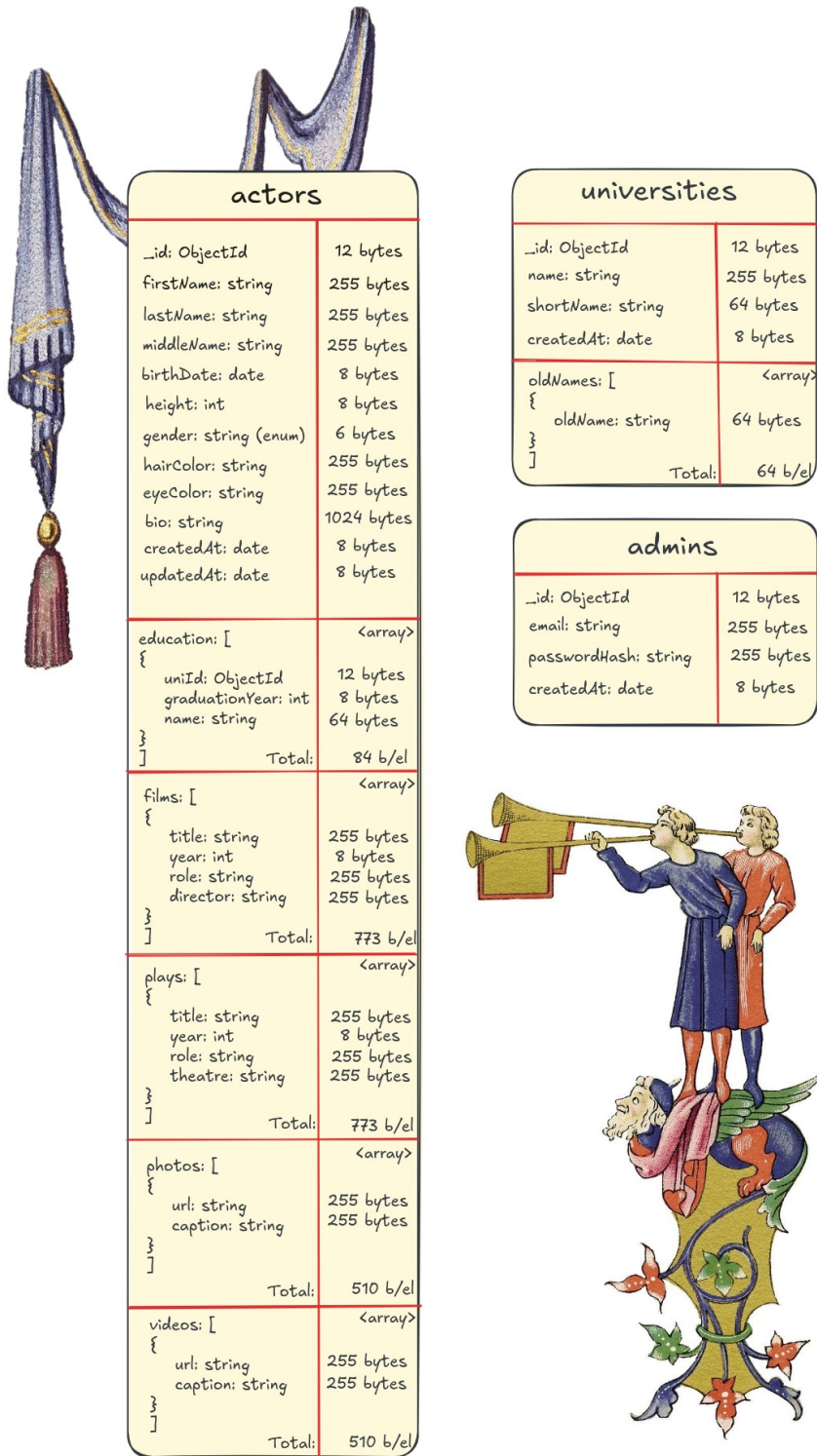


Рисунок 2 — графическое представление нереляционной базы данных

Описание назначений коллекций, типов данных и сущностей

В модели описаны три коллекции: **actors**, **admins** и **universities**. Коллекция **actors** содержит в себе документы с информацией о каждом актёре:

- уникальный идентификатор (встроенный тип ObjectId, занимает 12 байт),
- ФИО (три поля типа string. Считаем, что каждое занимает не более 255 байт),
- birthDate - дата рождения актёра типа date, занимает 8 байт,
- height - рост актёра - 8 байт, gender - enum (каждое возможное значение занимает не более 6 байт),
- цвет волос и глаз - строковые поля длиной не более 255 байт каждое,
- биография до 1024 байт,
- массив вузов, каждый элемент которого до 84 байт (12 на идентификатор, 8 на год выпуска, 64 на строку с сокращенным названием),
- информация о фильмах и спектаклях, которая занимает от 1546 байт суммарно (по 773 на каждый фильм и спектакль),
- фото и видео по 510 байт каждое (два поля string, каждое до 255 байт),
- атрибуты времени: дата создания и редактирования записи на 16 байт вместе.

Коллекция **admins** хранит информацию об администраторах платформы:

- идентификатор на 12 байт,
- уникальный адрес почты (строка до 255 байт),
- хэш пароля, тоже строка на 255 байт,
- и дата создания записи (8 байт).

Коллекция **universities** хранит информацию о старых названиях университетов (но потенциально легко расширяема):

- идентификатор на 12 байт,
- полное текущее название на 255 байт,
- сокращённое название на 64 байт,
- массив со вложенными структурами на 255 байт.

Оценка объема информации, хранимой в модели

Наибольшую сложность в оценке представляют данные, хранящиеся в массивах. Они будут рассмотрены далее отдельно. Базу составляют константные значения - все поля, имеющие фиксированный размер:

$$\text{modelSizeBase} = \text{_id}(12) + \text{firstName}(255) + \text{lastName}(255) + \text{middleName}(255) + \text{birthDate}(8) + \text{height}(8) + \text{gender}(6) + \text{hairColor}(255) + \text{eyeColor}(255) + \text{bio}(1024) + \text{createdAt}(8) + \text{updatedAt}(8) = 3373 \text{ (bytes)}.$$

Необходимо добавить следующие данные:

$$\text{sizeEdu} = K_{\text{uni}} * 84 \text{ bytes};$$
$$\text{sizeFilms} = K_{\text{films}} * 773 \text{ bytes};$$
$$\text{sizePlays} = K_{\text{plays}} * 773 \text{ bytes};$$
$$\text{sizePhotos} = K_{\text{photos}} * 510 \text{ bytes};$$
$$\text{sizeVideos} = K_{\text{videos}} * 510 \text{ bytes};$$

На основе статистических данных были получены следующие константы:

$$K_{\text{uni}} = 1;$$
$$K_{\text{films}} = 73;$$
$$K_{\text{plays}} = 19;$$
$$K_{\text{photos}} = 21;$$
$$K_{\text{videos}} = 1;$$

Исходя из этого:

$$\text{sizeEdu} = 84 \text{ bytes};$$
$$\text{sizeFilms} = 56429 \text{ bytes};$$
$$\text{sizePlays} = 14687 \text{ bytes};$$
$$\text{sizePhotos} = 10710 \text{ bytes};$$
$$\text{sizeVideos} = 510 \text{ bytes};$$

Итого на хранение одного актёра в среднем требуется 85793 байт. Количество вузов можно выразить через количество актёров как:

$$\text{uniCount}(x) = x * K_{\text{uni}} / \text{ActPerUni} = x / 97 = 0.01 * x;$$

Каждый в среднем менял название два раза. Отсюда добавляется $8.41 * N$. Количество администраторов фиксировано и равно 3. Отсюда ещё 1590 байт на коллекцию администраторов.

Итоговая формула зависимости размера модели от количества записей с актёрами:

$$\text{modelSize}(x) = 85801.41 * x + 1590$$

Зависимость линейная.

Примеры запросов

1. Просмотр лендинга и переход к поиску

Здесь требуется только получить данные для главной страницы

```
db.actors.find(  
  {},  
  {  
    "firstName": 1,  
    "lastName": 1,  
    "middleName": 1,  
    "photos": { $slice: 1 },  
    "films": { $size: 1 }  
  }  
)  
.limit(10)
```

Запрос требуется один. При увеличении числа актеров время выполнения остается константным $O(1)$, так как используется `limit(10)`, и MongoDB может остановить сканирование после нахождения 10 документов. Задействована только коллекция "Actors".

2. Поиск профилей

```
db.actors.find({
```

```

$and: [
  { $text: { $search: "Лабыбы" } },
  { "gender": "male" },
  {
    "birthDate": {
      $gte: new Date("1970-01-01"),
      $lte: new Date("1990-12-31")
    }
  },
  { "height": { $gte: 170 } },
  {
    "education": {
      $elemMatch: {
        "uniId": ObjectId("1234")
      }
    }
  }
]
}))
.sort({ "lastName": 1, "firstName": 1 })
.limit(20)

```

Запрос один. Коллекция одна - actors. Увеличение количества актёров не приведёт к увеличению числа запросов. Сложность с индексами - $O(\log N)$ по каждому условию, без индексов - $O(N)$ — полное сканирование коллекции.

3. Просмотр профиля

```

db.actors.findOne(
  { "_id": ObjectId("5476") },
  {
    "firstName": 1,

```

```
"lastName": 1,  
"middleName": 1,  
"birthDate": 1,  
"height": 1,  
"gender": 1,  
"hairColor": 1,  
"eyeColor": 1,  
"bio": 1,  
"education": 1,  
"films": 1,  
"plays": 1,  
"photos": 1,  
"videos": 1  
}  
)
```

Один запрос, одна коллекция, при увеличении количества актёров время поиска увеличивается несущественно в виду индексации и оптимизаций поиска по ObjectID. Поиск по `_id` в MongoDB имеет сложность $O(\log N)$ благодаря B-дереву.

Реляционная модель данных.

Графическое представление

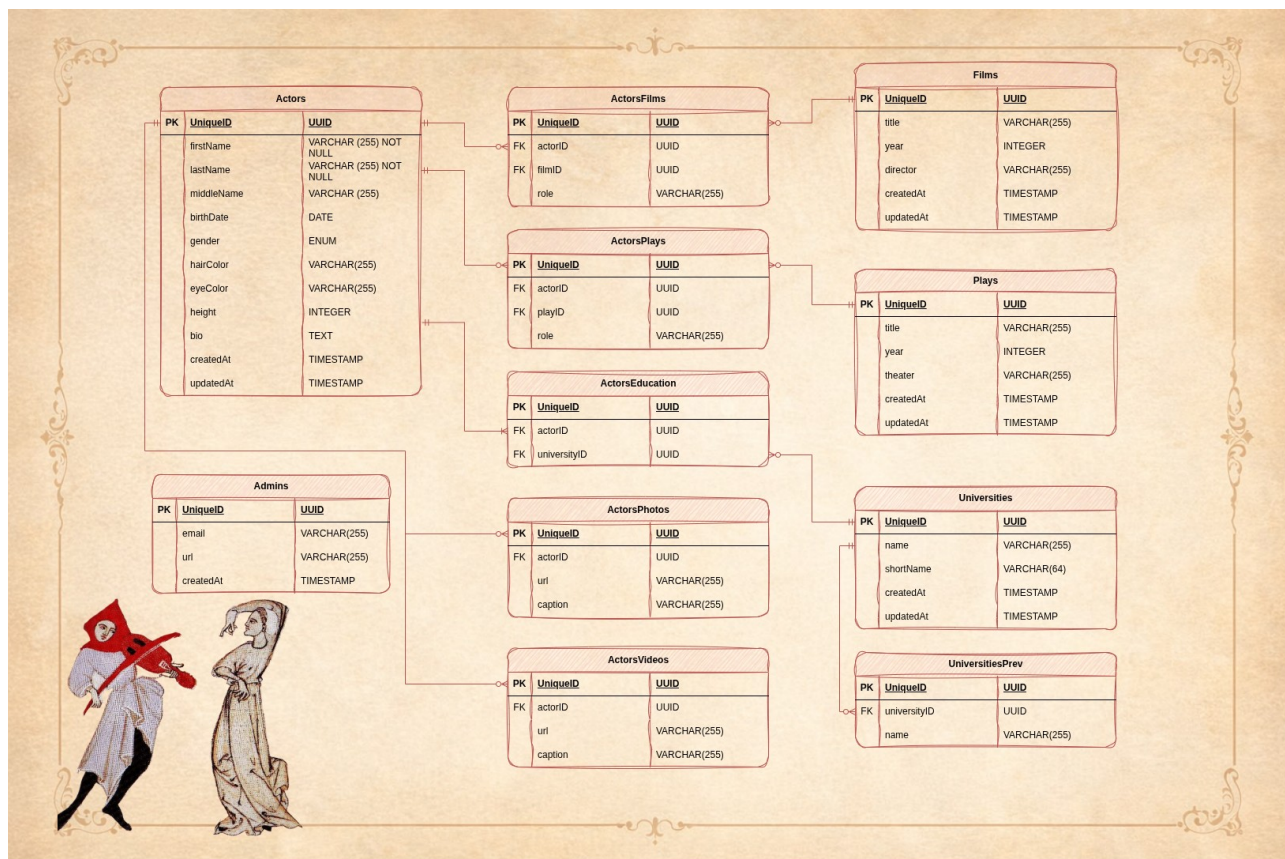


Рисунок 3 — графическое представление реляционной базы данных

Описание назначений коллекций, типов данных и сущностей

База данных состоит из 11 таблиц: Actors, Films, Plays, Universities, Admins, UniversitiesPrev, ActorsFilms, ActorsPlays, ActorsEducation, ActorsPhotos, ActorsVideos.

- Actors

Таблица с основной информацией об актёрах.

Хранит уникальный id (16 байт в PostgreSQL), firstName, lastName, middleName (ФИО по 255 байт - всего 765 байт), birthDate - дата рождения актёра типа DATE (4 байта), gender - до 6 байт, цвет волос и глаз - VARCHAR поля по 255 байт, bio типа TEXT (может хранить данные размером до Гб, но при расчетах был принят размер в 1024 байт по аналогии с нереляционной БД), timestamp создания и редактирования записи - два поля по 8 байт каждое).

- Films

Таблица с фильмами.

Уникальный id (16 байт), title - название фильма в 255 байт, year - год (integer 4 байт), режиссёр картины - 255 байт, timestamps аналогично таблице с актёрами (в сумме 16 байт).

- Plays

Таблица со спектаклями

Уникальный id (16 байт), title - название фильма в 255 байт, year - год (integer 4 байт), театр - 255 байт, timestamps аналогично предыдущим таблицам (в сумме 16 байт).

- Universities

Таблица с вузами.

Хранит информацию об университетах: идентификатор на 16 байт, полное текущее название на 255 байт, сокращённое название на 64 байт, timestamps на 16 байт.

- UniversitiesPrev

Соответствие id вуза с его предыдущими названиями.

Хранит UUID (16b), universityID (16 байт) - для связи с таблицей universities, name на 255 байт.

- Admins

Хранит информацию об администраторах веб-приложения.

UUID(16b), email (почта хранится в виде строки VARCHAR(255)), passwordHash размером 255 байт и timestamp создания записи на 8 байт.

- ActorsFilms

Описывает связь таблиц Actors и Films.

У каждой записи отсюда обязательно есть id актёра и фильма, но не у каждого актёра обязательно есть соответствующая ему запись в этой таблице.

Хранит UUID, actorID, filmID - поля по 16 байт, всего 48. Поле role на 255 байт.

- ActorsPlays

Описывает связь таблиц Actors и Plays.

У каждой записи отсюда обязательно есть id актёра и спектакля, но не у каждого актёра обязательно есть соответствующая ему запись в этой таблице.

Хранит UUID, actorID, playID - поля по 16 байт, всего 48. Поле role на 255 байт.

- ActorsEducation

Описывает связь Actors и Universities. Здесь связь аналогична предыдущим.

UUID (16b), actorID тоже занимает 16 байт, uniID аналогично.

- ActorsPhotos

Для хранения фото актёров.

Хранится UUID (16b), actorID (16b), url и caption на 255 байт каждое.

- ActorsVideos

Для хранения видео актёров.

Хранится UUID (16b), actorID (16b), url и caption на 255 байт каждое.

Оценка объема информации, хранимой в модели

В этих расчётах также используются константы:

$K_{uni} = 1;$

$K_{films} = 73;$

$K_{plays} = 19;$

$K_{photos} = 21;$

$K_{videos} = 1;$

Они позволяют оценить размер связующих таблиц, но их недостаточно, чтобы оценить объём таблиц Films, Plays (Universities был исследован в предыдущем разделе). Введём дополнительные константы:

ActorsPerFilm = 21;

ActorsPerPlay = 13;

Размер одной записи в Films равно $16+255+4+255+8+8=546$. Тогда размер таблицы Films:

$FilmsSize(x) = x * K_films / ActorsPerFilm * 546 = 1898x$

Аналогично можно получить размер Plays:

$PlaysSize(x) = x * K_plays / ActorsPerPlay * 546 = 798x$

Для университетов:

$uniCount(x) = x * K_uni / ActPerUni * 351 = 351 * x / 97 = 3.619 * x$;

Для таблицы Actors:

$ActorsSize(x) = 2341x$

ActorsFilms:

$ActorsFilmsSize(x) = 303 * 73 * x = 22119x$

ActorsPlays:

$ActorsPlaysSize(x) = 303 * 19 * x = 5757x$

ActorsEducation:

$ActorsEducationSize(x) = 48x$

ActorsPhotos:

$ActorsPhotos(x) = 542 * 21 * x = 11382x$

ActorsVideos:

$ActorsVideos(x) = 542x$

UniversitiesPrev:

$UniversitiesPrevSize(x) = 287 * 2 * 3.619 * x = 2077.306x$

Число администраторов считаем константным = 3. С учетом размера записи администратора объём - $534 * 3 = 1602$ байта.

Тогда полный размер модели:

$modelSize(x) = 46965,925 * x + 1602$.

Примеры запросов

1. Просмотр лендинга и переход к поиску

```
SELECT
    id,
    firstName,
    lastName,
    middleName
FROM Actors
LIMIT 10;
```

```
SELECT
    actorId,
    url,
    caption
FROM ActorsPhotos
WHERE actorId IN
ORDER BY actorId;
```

2 запроса к двум коллекциям. При увеличении объёма данных рост линейный

2. Поиск профилей

```
SELECT
    firstName, lastName, middleName,
    birthDate, height, gender,
    hairColor, eyeColor, bio
FROM Actors
WHERE id = '1234';
```

```
SELECT
```

```
ae.graduationYear,  
u.name as university_name,  
u.shortName as university_short_name  
FROM ActorsEducation ae  
JOIN Universities u ON ae.uniId = u.id  
WHERE ae.actorId = '1234';
```

```
SELECT  
f.title,  
f.year,  
f.director,  
af.role  
FROM ActorsFilms af  
JOIN Films f ON af.filmId = f.id  
WHERE af.actorId = '1234';
```

```
SELECT  
p.title,  
p.year,  
p.theatre,  
af.role  
FROM ActorsPlays af  
JOIN Plays p ON af.filmId = f.id  
WHERE af.actorId = '1234';
```

```
SELECT url, caption  
FROM ActorsPhotos  
WHERE actorId = '1234';
```

```
SELECT url, caption
FROM ActorsVideos
WHERE actorId = '1234';
```

6 запросов в 6 таблиц (или один большой JOIN).

Сравнение моделей.

Удельный объем информации.

Проведем вычисления для конкретного объема данных: 10000 актёров в базе.

NOSQL:

$\text{modelSize}(x) = 85801.41 * x + 1590$

size = 858015690 bytes

R = 1.00007

SQL:

$\text{modelSize}(x) = 46965,925 * x + 1602$

size = 469660852

R = 1.041

Таким образом, NoSQL занимает больше места, чем SQL. SQL экономит место за счет нормализации: данные о фильмах, спектаклях и связях не дублируются для каждого актера, а хранятся в отдельных таблицах. Зато в NoSQL все данные об актёре в одном месте.

Запросы по отдельным юзкейсам.

MongoDB:

- Все данные об актёре хранятся в одном документе
- Для получения полного профиля нужен 1 запрос
- Для лендинга с ограничением в 10 документов — 1 запрос
- Для сложного поиска — 1 запрос

Плюсы: минимальное количество запросов, все данные сразу доступны

Минусы: дополнительные расходы памяти

SQL:

- Для получения полного профиля нужно минимум 6 запросов
- Данные нормализованы, каждый факт хранится в одном месте

Плюсы: консистентность данных, экономия места

Минусы: множество JOIN'ов и подзапросов, сложность запросов

Выводы.

Для решения поставленной задачи NoSQL подходит больше, поскольку обеспечивает быстрое чтение и не требует частого обновления данных из других коллекций.

4. Разработанное приложение

Краткое описание и технологический стек

Было реализовано полнофункциональное веб-приложение для управления базой данных актёров театра и кино.

Система позволяет:

- Вести подробные профили актёров (биография, фото, роли, образование)
- Выполнять сложный поиск с фильтрацией
- Загружать и хранить медиафайлы
- Просматривать статистику
- Импортировать/экспортировать данные

Технологический стек строится вокруг трёх контейнеризованных сервисов:

Frontend — Angular

Backend — Kotlin + Spring Boot

DB — MongoDB

Всё вместе поднимается через docker-compose.

Клиентская часть реализована в виде приложения на базе Angular.

Она включает в себя следующие модули:

- Каталог актёров — отображение списка профилей с поддержкой пагинации.
- Карточка актёра — подробная информация об артисте, биография, фотографии, список ролей, контактная информация, театры, звания, годы активности.
- Форма поиска — фильтрация с множественным выбором критериев. Можно фильтровать по ФИО, полу, возрасту, росту, весу, годам активности, вузу, театру, званию, цвету волос, глаз и жанру.

- Панель администратора — позволяет создавать и редактировать карточки, добавлять вузы, смотреть статистику и импортировать/экспортировать данные.

Бэкенд предоставляет REST API, реализующий:

- CRUD-операции над профилями актёров
- Эндпоинты для фильтрации и поиска по критериям
- Загрузку, хранение и удаление медиафайлов (фотографии)
- Аутентификацию и авторизацию администраторов
- Документация API генерируется автоматически в формате OpenAPI (Swagger UI).

Модель базы данных была подробно рассмотрена в предыдущем разделе.

Кроме того, было обеспечено хорошее тестовое покрытие — было реализовано 170 тестов бэкенда.

Снимки экрана приложения.

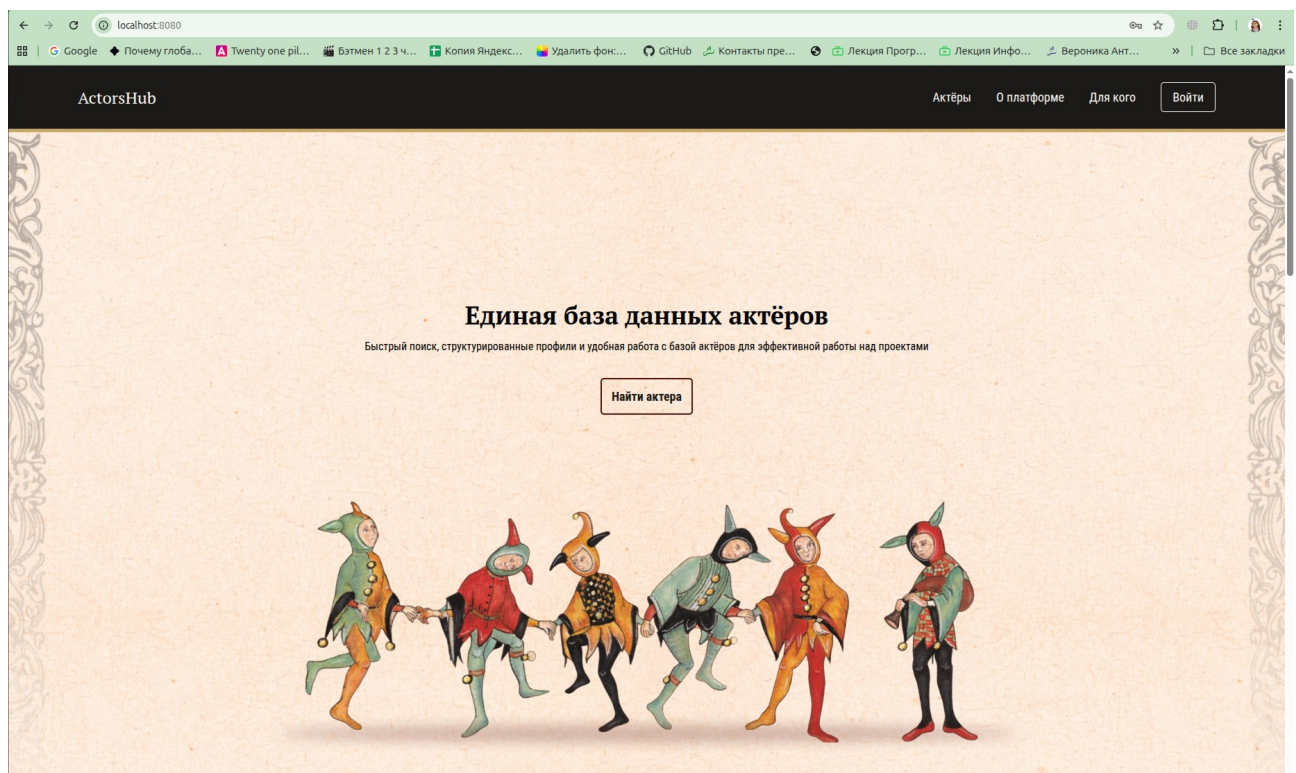


Рисунок 4 — главная страница приложения

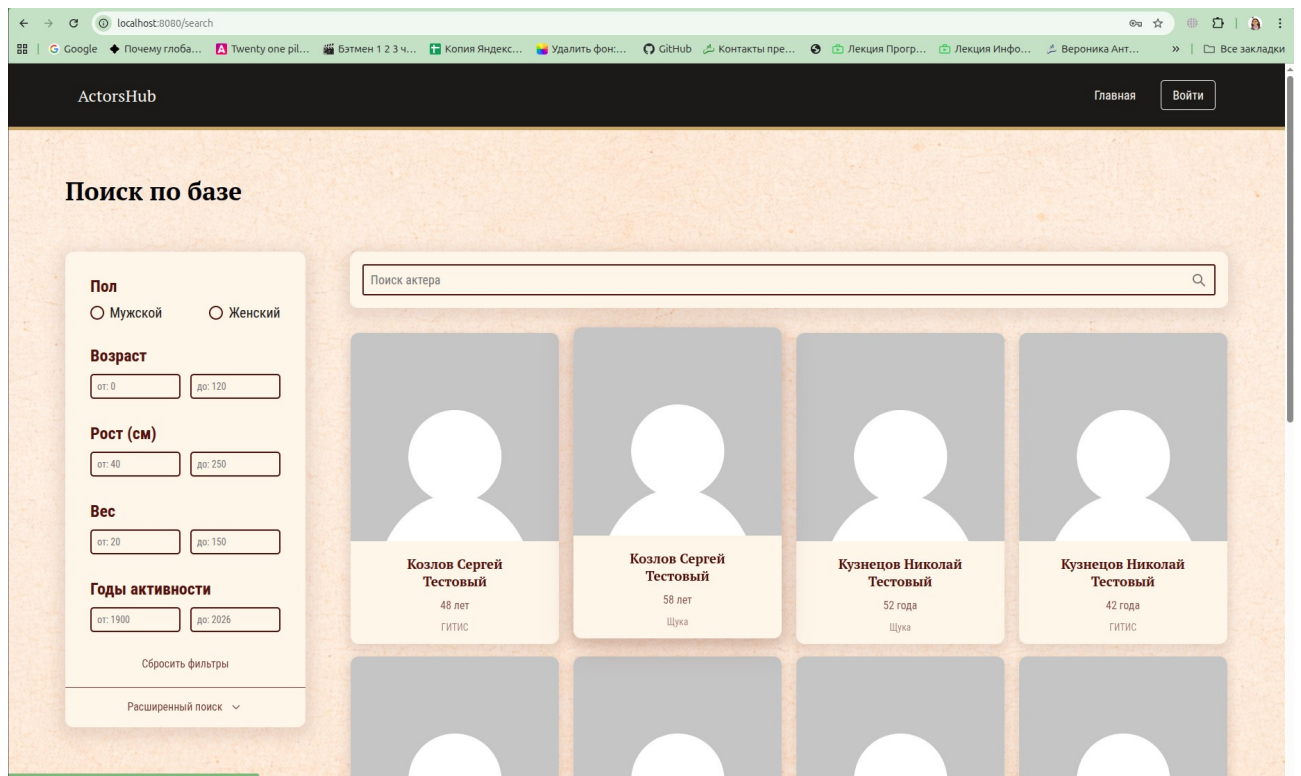


Рисунок 5 — каталог актёров

5. Выводы

Достигнутые результаты

В результате работы было реализовано веб-приложение, позволяющее осуществлять поиск актёров в базе данных, просматривать информацию о них, при наличии прав администратора редактировать сущности и просматривать статистику. В приложении поддерживается массовый импорт/экспорт.

Недостатки и пути для улучшения полученного решения

Несмотря на то, что заявленный функционал приложение реализует в полном объёме, глобально ему не хватает гибкости: так, карточки может создавать только пользователь с правами администратора. Возможно, следует добавить возможность обычному пользователю создать визитку и, например, реализовать систему модерации, чтобы администраторы одобряли размещение на сайте.

Будущее развитие решения

В будущем можно расширить ряд предоставляемых возможностей: создание карточки актёра простым пользователем, возможность связаться с актёром напрямую через приложение, каталог кастингов и система уведомлений о появившихся ролях интересующих жанров для актёров.

6. Приложения

Документация по сборке и развертыванию приложения

Необходимо выполнить следующие действия:

1. Клонировать репозиторий <https://github.com/moevm/nsql1h26-actor>
2. Зайти в директорию проекта
3. Из корня выполнить

```
docker compose build --no-cache && docker compose up -d
```

Разработанное приложение будет доступно на <http://localhost:8080>

7. Литература

Документация по сборке и развертыванию приложения

Была использована следующая литература:

1. Документация к MongoDB: <https://www.mongodb.com/docs/manual/>
2. Исследование о киноиндустрии: <https://asi.ru/news/204298/>

Ссылка на репозиторий

<https://github.com/moevm/nsql1h26-actor>